Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Name: Syed Murtaza

Hall ticket no: 2303A51259

Batch no: 19

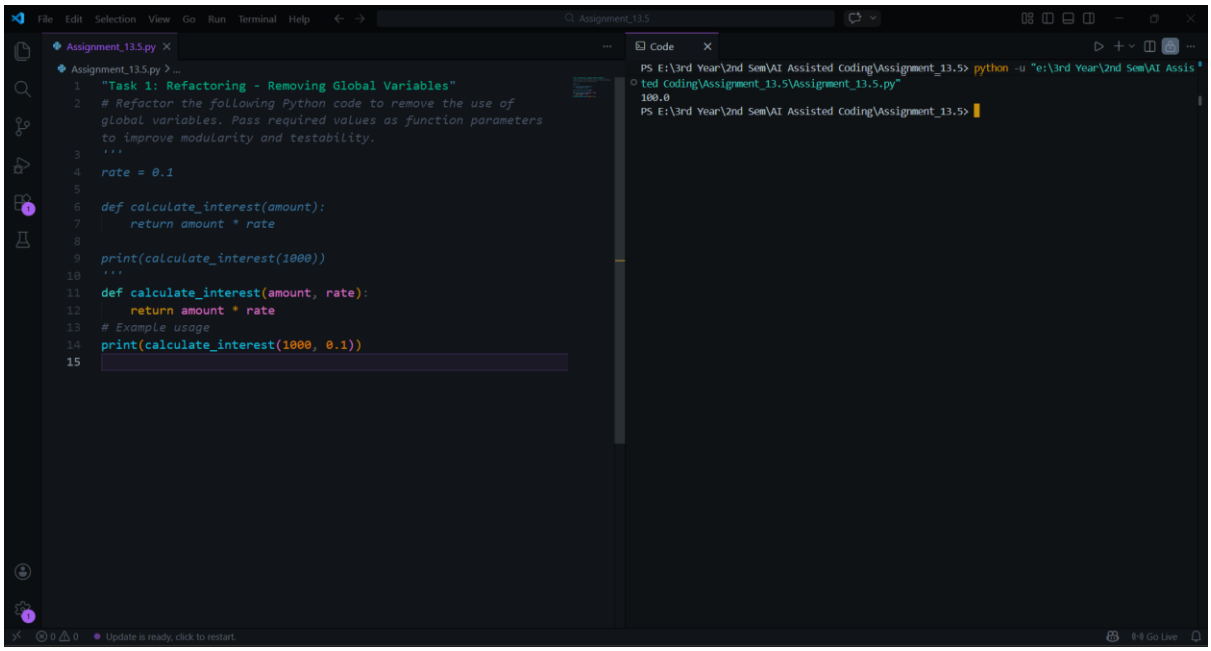
Task 1: Refactoring – Removing Global Variables

Prompt:

Refactor the following Python code to remove the use of global variables. Pass required values as function parameters to improve modularity and testability.

```
rate = 0.1
def calculate_interest(amount):
    return amount * rate
print(calculate_interest(1000))
```

Code & Output:



Explanation:

The original code used a global variable rate, which reduces modularity. The refactored version passes rate as a function parameter, making the function independent of global state. This improves testability and reusability.

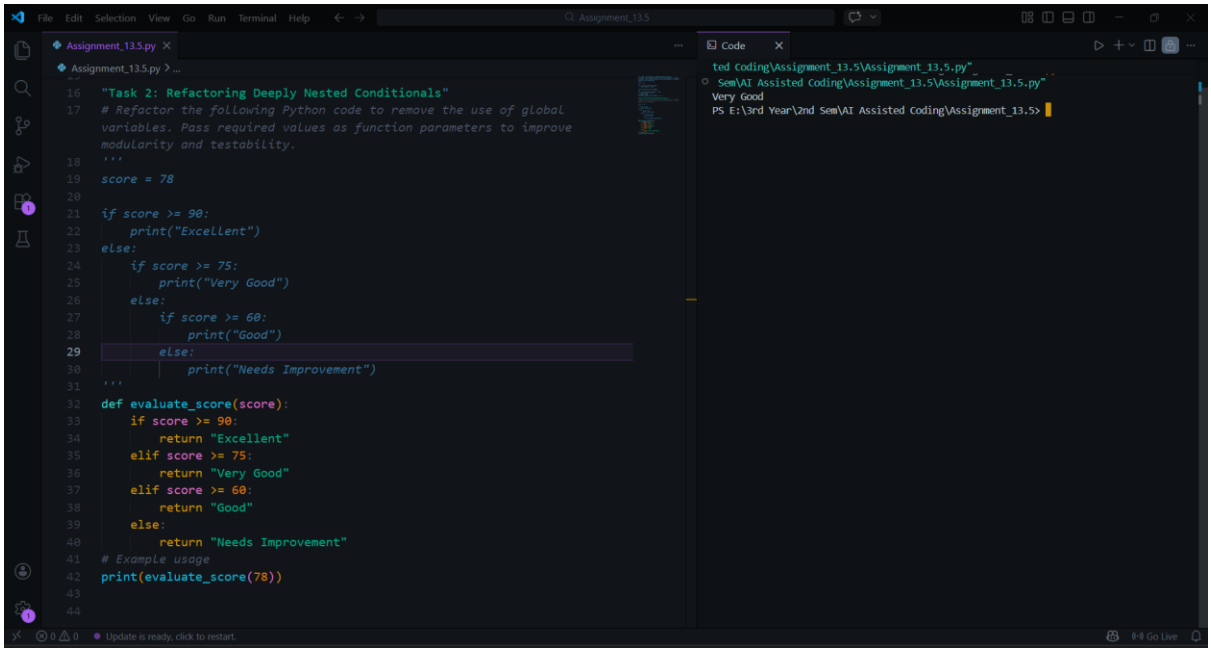
Task 2: Refactoring Deeply Nested Conditionals

Prompt:

Refactor the nested conditional statements into a cleaner and more readable structure.

```
score = 78
if score >= 90:
    print("Excellent")
else:
    if score >= 75:
        print("Very Good")
    else:
        if score >= 60:
            print("Good")
        else:
            print("Needs Improvement")
```

Code & Output:



Explanation:

The legacy code contains multiple nested if statements, which makes the control flow harder to read. The refactored version replaces nested conditions with a flat if-elif-else structure. This simplifies the logic and improves readability. It also follows standard Python coding practices for writing clear conditional statements.

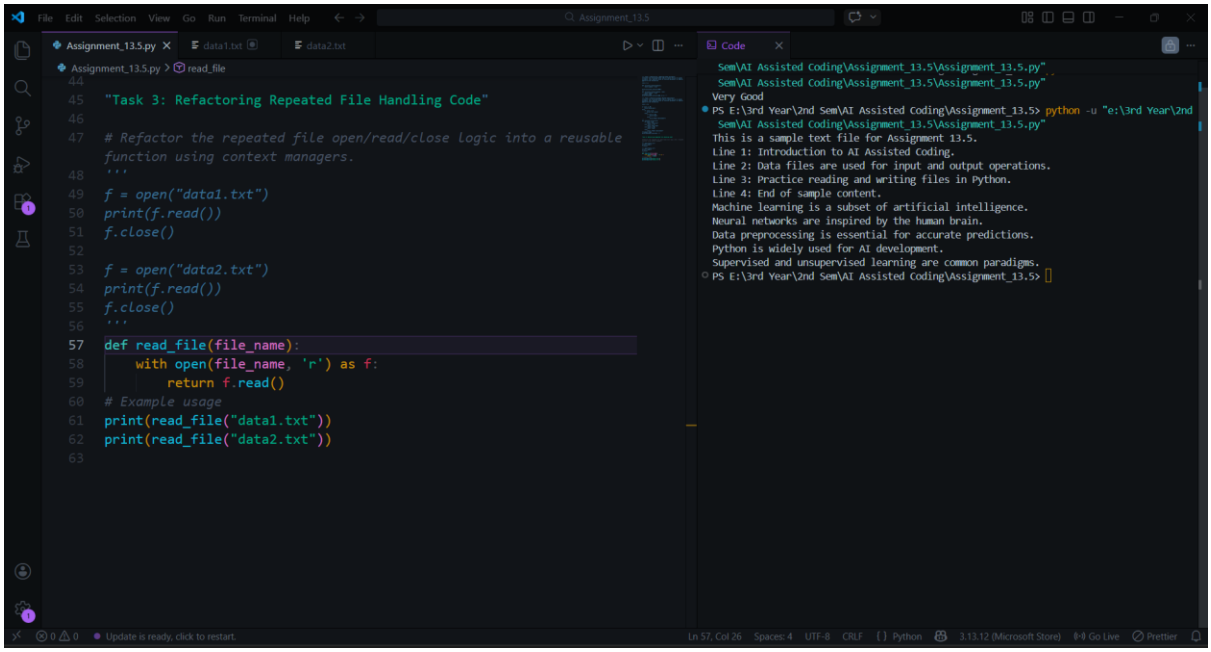
Task 3: Refactoring Repeated File Handling Code

Prompt:

Refactor the repeated file open/read/close logic into a reusable function using context managers.

f = open("data1.txt")
print(f.read())
f.close()
f = open("data2.txt")
print(f.read())
f.close()

Code & Output:



Explanation:

The original code repeatedly opens and closes files, which results in duplicated code. The refactored solution introduces a reusable function and uses the with open() context manager. This ensures that the file is automatically closed after use. It improves maintainability and follows the DRY principle.

Task 4: Optimizing Search Logic

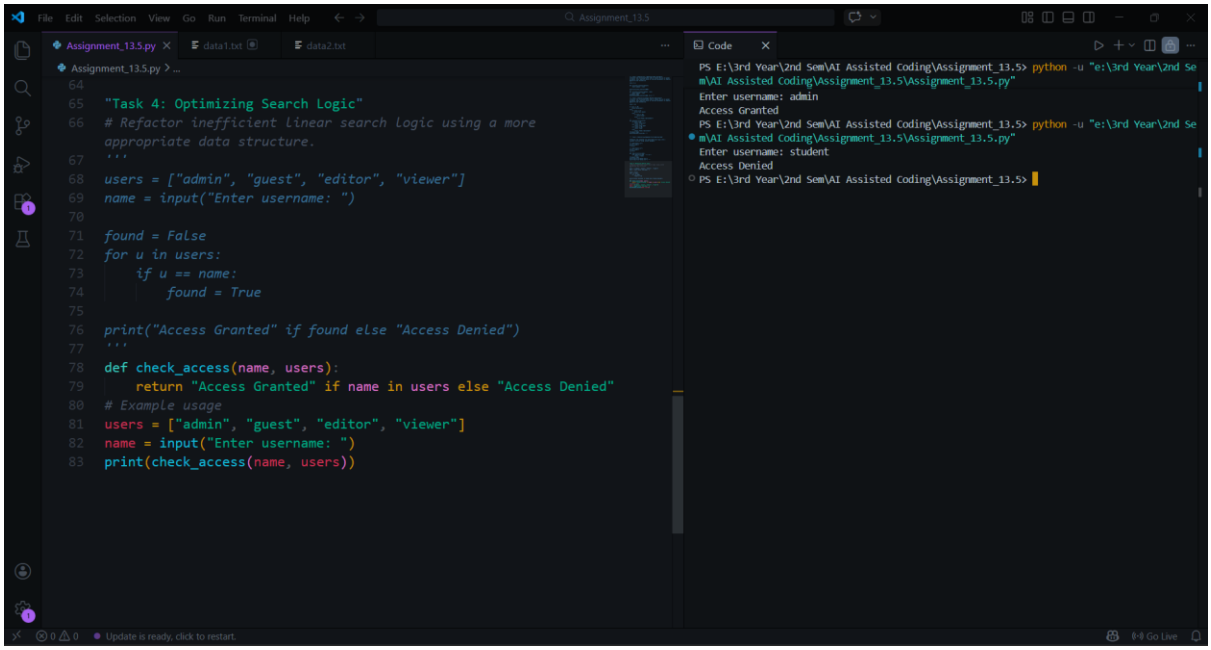
Prompt:

Refactor inefficient linear search logic using a more appropriate data structure.

users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:

found = True
print("Access Granted" if found else "Access Denied")

Code & Output:



Explanation:

The legacy code uses a loop to search through a list, which requires checking each element sequentially. This results in linear time complexity. The refactored version uses a set data structure, which allows constant-time membership checking. This improves performance and makes the code simpler.

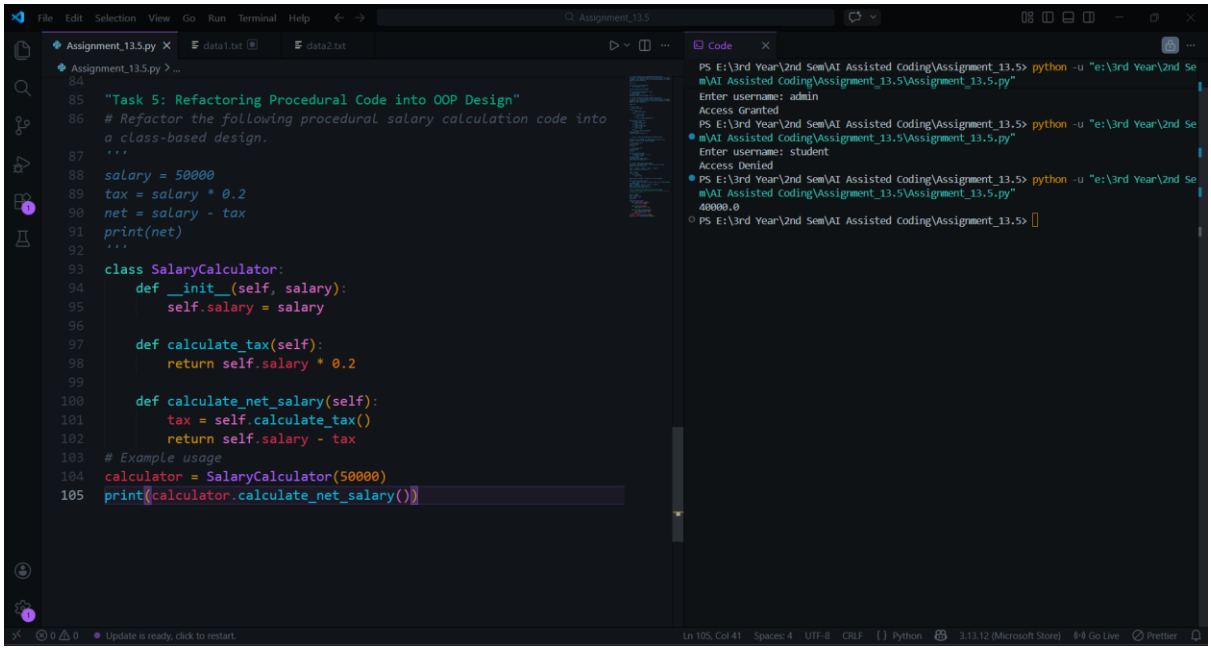
Task 5: Refactoring Procedural Code into OOP Design

Prompt:

Refactor the following procedural salary calculation code into a class-based design.

salary = 50000
*tax = salary * 0.2*
net = salary - tax
print(net)

Code & Output:



Explanation:

The legacy code performs calculations using simple procedural statements without structure. The refactored version introduces a class that encapsulates salary and tax calculations. This improves code organization and allows the logic to be reused easily. It also demonstrates object-oriented programming principles such as encapsulation.

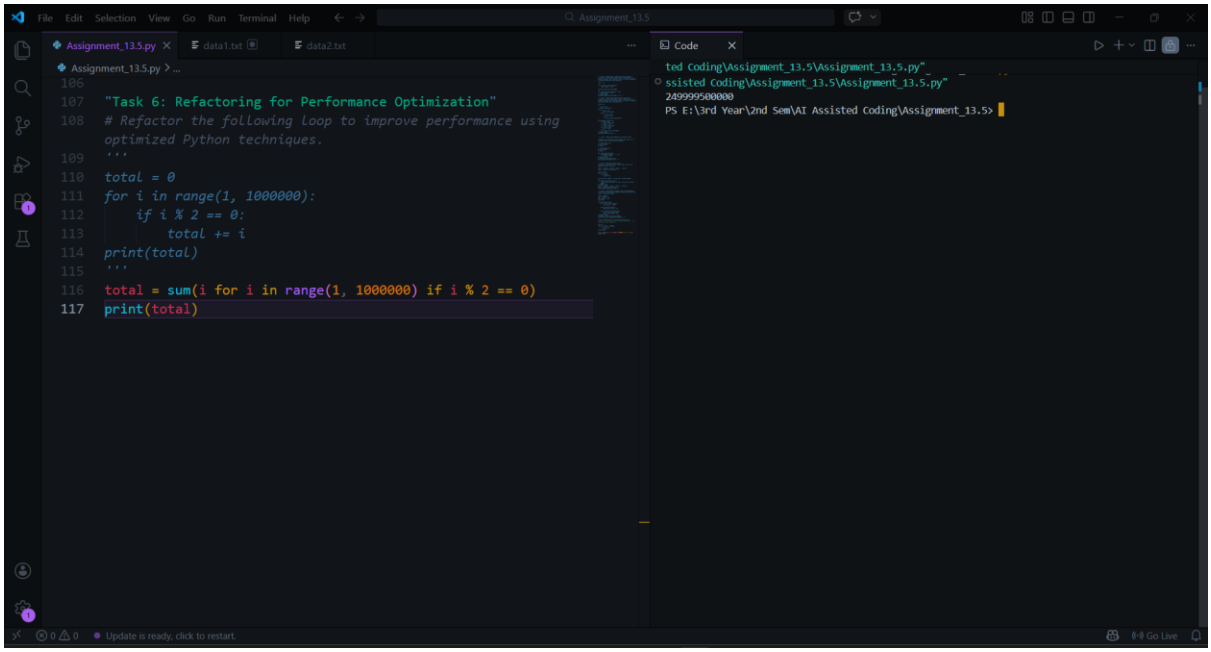
Task 6: Refactoring for Performance Optimization

Prompt:

Refactor the following loop to improve performance using optimized Python techniques.

```
total = 0
for i in range(1, 1000000):
    if i % 2 == 0:
        total += i
print(total)
```

Code & Output:



Explanation:

The original code manually iterates through a large range and checks each value. The refactored version uses Python's built-in sum() function with a generator expression. This approach reduces the number of lines and improves readability. It also takes advantage of optimized built-in functions.

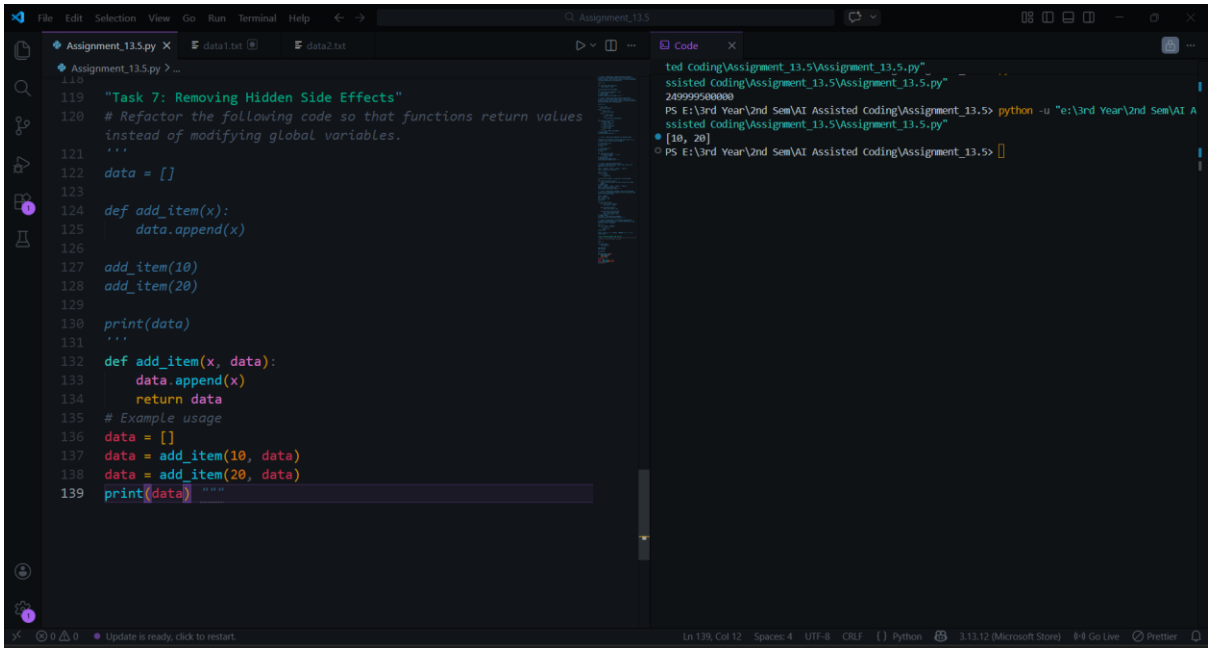
Task 7: Removing Hidden Side Effects

Prompt:

Refactor the following code so that functions return values instead of modifying global variables.

```
data = []
def add_item(x):
    data.append(x)
add_item(10)
add_item(20)
print(data)
```

Code & Output:



Explanation:

The legacy function modifies a global list directly, which creates hidden side effects. This can lead to unexpected behavior in larger programs. The refactored version returns a new list instead of modifying global state. This functional approach improves predictability and makes the function easier to test.

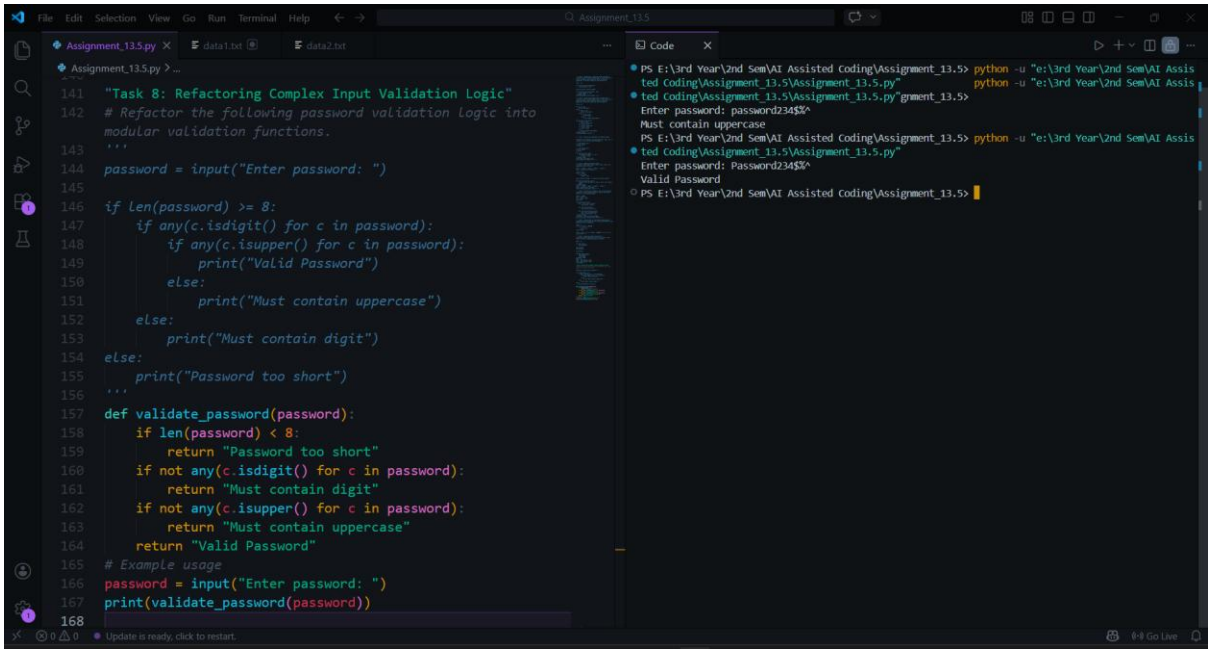
Task 8: Refactoring Complex Input Validation Logic

Prompt:

Refactor the following password validation logic into modular validation functions.

```
password = input("Enter password: ")
if len(password) >= 8:
    if any(c.isdigit() for c in password):
        if any(c.isupper() for c in password):
            print("Valid Password")
        else:
            print("Must contain uppercase")
    else:
        print("Must contain digit")
else:
    print("Password too short")
```

Code & Output:



Explanation:

The original code contains multiple nested checks that make the logic harder to read. The refactored version separates each validation rule into its own function. This modular structure improves readability and makes testing each rule easier. It also allows new validation rules to be added without modifying the entire structure.

Final Conclusion:

This lab demonstrated how AI tools can assist in refactoring legacy Python code to improve readability, modularity, and performance. Techniques such as removing global variables, simplifying nested conditions, optimizing search logic, and applying object-oriented design were used. These improvements make the code easier to understand, maintain, and extend while preserving its original functionality.